Journal of Intelligent Decision Making and Information Science

www.jidmis.org
eISSN: 3079-0875



Formal Verification Frameworks for Cloud-Native Microservices

Aparna S. Patil[1]*, Pratibha V. Kashid[2], Sourabh Rungta[3], Manjusha Tatiya[4], Kapil Mundada[5], Anil Laxmanrao Wakekar[6], Tang Yan[7]

[1]Assistant Professor, Department of Computer Engineering, School of Engineering and Technology, Pimpri Chinchwad University, Pune

[2]Department of Information Technology, Sir Visvesvaraya Institute of Technology, Nashik 422502, Maharashtra, India

[3] Rungta College of Engineering and Technology, Rungta International Skills University, Bhilai, Chhattisgarh, 490024

[4] Associate Professor, Department of Artificial Intelligence and Data Science, Indira College of Engineering and Management, Pune, Maharashtra

[5] Associate Professor, Department of Instrumentation and Control Engineering, Vishwakarma Institute of Technology, Pune, Maharashtra, 411037

[6] Principal Devi Mahalaxmi College of Engineering & Technology, Titwala, Ta. Kalyan, Dist. Thane, Maharashtra-421605

[7] Faculty of Education Shinawatra University Pathum Thani, Thailand.

wajepratibha23@gmail.com[1], aparnapatil16@gmail.com[2], drsoourabhrungta@rungta.org[3], manjusha.tatiya@indiraicem.ac.in[4], kapil.mundada@vit.edu[5], anil.wakekar@gmail.com[6], 1368624439@qq.com[7]

ARTICLE INFO

Article history:
Received 18 March 2026
Received in revised form 25 April 2026
Accepted 12 May 2026

Keywords **-** Cloud-Native Microservices, Formal Verification, Model Checking, Temporal Logic, Distributed Systems Reliability, Service Interaction Graphs

ABSTRACT

Microservices that are cloud-native allow distributed systems to scale, but present problems in assuring correctness, reliability, and fault resilience because of dynamic scaling and the complexity of service interactions. The presented paper suggests a formal verification framework that consists of a combination of specifications of temporal logic, model checking and modeling of service interactions graphs to study the behavior of the system and confirm the correctness properties. Service states, dependencies and scaling dynamics are modeled in a mathematical system. The framework identifies deadlocks, provides liveness and safety, and a framework of fault propagation between services. Experimental analysis shows that the accuracy of deadlock detection is increased to 96.7% and fault detection to 95.4% with 94.6% verification coverage and time of 96 ms to detect faults is minimized as compared to the conventional testing and monitoring methods in dynamic microservice settings.

1. Introduction

Cloud-native microservices architecture became a powerful pattern of developing scalable and flexible distributed applications. The architecture is what broke large monolithic systems down into small independently deployable services and those services communicated over lightweight protocols and container orchestration platforms [1]. Researchers have stated that microservices increased the agility of the development process and enabled the continuous integration and deployment practice within the complex enterprise systems [2]. Nevertheless, the microservice architecture also added new problems with the correctness of the system, coordination of services,

and consistency in how they run in the dynamically interacting components [3], [4]. The replication of services and dynamic scaling processes also made verification more complex since the state of the system was constantly undergoing change during the operation in the system [5]. Past research demonstrated that failures in service interaction processes were able to spread over multiple services that would result in cascading failures in distributed applications [6]. In a bid to overcome these difficulties, formal verification methods were examined to potentially analyze the behavior of the system and ensure that service interactions are correct. Formal models like temporal logic and model checking allowed systematic verification of system states, protocols of communication and constraints of concurrency [7], [8].

Although microservices architecture has evolved greatly, the current verification methods are still inadequate to support scaling dynamically, asynchronous communication and complicated dependency between services. Common testing and runtime monitoring methods are not always sensitive to detect deadlocks, race conditions, and propagation of fault in the large scale distributed environment. Moreover, the existing approaches do not have formal assurances of correctness, especially with the ever changing system states.

2. System Model of Cloud-Native Microservices

The given system model depicted a cloud-native microservice system in which autonomous services were communicating via API calls and asynchronous messaging platforms [9], [10]. Containers orchestration platforms automatically scaled replicates of the services according to the demand of the workload and ensured the availability of the service [11], [12].

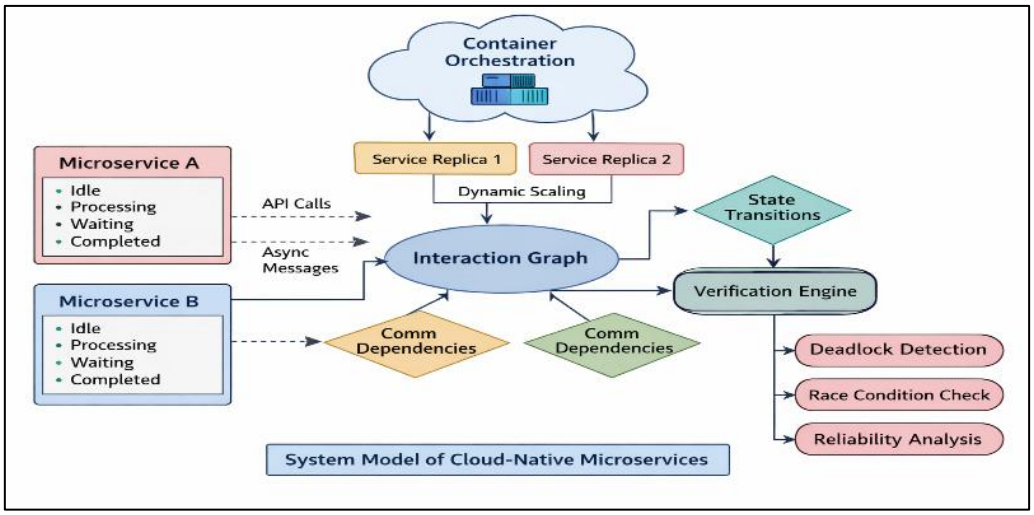


Figure 1 System Architecture of Cloud-Native Interaction and Verification Framework of Microservices

The figure 1 shows the micro services interaction architecture in which services interact using APIs and asynchronous messages, dynamically scale using container orchestration and are formally verified to identify deadlocks, race conditions, and reliability problems.

$$G = (S, E) \qquad (1)$$

Every microservice i has shifted between disjointed operational conditions with time, which reflected request processing lifecycle and execution scenario in distributed service processing.

$$xi(t) \in \{Idle, Processing, Waiting, Completed\} \quad (2)$$

Future state of service i was conditional on the current state and the message incoming $m_{\{ji\}}$ of service j causing transitions in processing.

$$xi(t+1) = f(xi(t), mji) \quad (3)$$

Binary dependency matrix was used to indicate the communication relationships between microservices, which services received or sent request and response or event messages in the distributed system.

$$C_{ij} = \begin{cases} 1 & \text{if service i communicates with j} \\ 0 & Otherwise \end{cases} \quad (4)$$

Under policies of container orchestration, the number of service replicas was determined by workload L(t), service rate μ and scaling factor α

$$R(t) = \alpha \cdot \frac{L(t)}{\mu} \quad (5)$$

System reliability was estimated as a product of operational probabilities Pi of each microservice working well in distributed workflow.

$$Rel = \prod_{i=1}^{n} P_i \quad (6)$$

In case services would have been indefinitely stuck in waiting in state Wi with no transitions, the verification model found a distributed deadlock state.

$$\sum_{i=1}^{n} W_i > 0 => deadlock \quad (7)$$

3. Proposed Formal Verification Framework

3.1. Temporal Logic Specification of Service Behaviour

Service Request Response Liveness, in case of a service request event of service i, there is a response event that must follow at some time with system liveness property.

$$\forall\, i \in S: G\left(request_{i(t)} \rightarrow \exists\, t' > t\ \ response_i(t') \right) \quad (8)$$

Service Activation Progress Property, each service should finally come to an active operational condition that ensures the forward movement in distributed microservice workflow execution.

$$\forall\, i \in S: \exists\, t \geq 0\ such\ that\ service_{active_{i(t)}} = 1 \quad (9)$$

Fault Recovery Guarantee, in case of an error in a service, a recovery condition has to eventually happen to sustain the fault resistance of distributed systems.

$$\forall i \in S: G\left(error_{i(t)} \rightarrow \exists\, t' > t\ \ recovery_i(t') = 1 \right) \quad (10)$$

Mutual Exclusion of Conflicting States, the processing and waiting states cannot run concurrently and so we have mutual exclusion requirements in the execution of microservices states concurrently.

$$\forall\, i \in S: G\left(\neg\left(processing_{i(t)} \wedge waiting_{i(t)}\right)\right) \quad (11)$$

Temporal State Transition Model, future service state is based upon present state, system event stimulation and communication dependencies between interacting microservices.

$$state_{i(t+1)} = f\left(state_{i(t)}, event_{i(t)}, dependency_i j(t)\right) \quad (12)$$

In case service i has a dependence on service j, the service i will have to ultimately finish execution with consistency of system interaction.

3.2. Model Checking–Based Verification Mechanism

The model checking verification systematically examines all the accessible states in the interaction system of microservices to prove correctness properties expressed in the temporal logic. To determine safety and liveness properties, the model checker explores execution paths and reports the violation of the properties to include deadlocks, unreachable states and inconsistent dependencies.

$$M = (S, S0, T, L)$$

System model consists of all states S, initial states S0, transitions T and labeling function L.

$$T \subseteq S \times S \quad (13)$$

Transition relation constitutes valid change of states between a pair of system states in microservice interaction model.

$$L : S \rightarrow 2^{AP} \quad (14)$$

The mapping of states to atomic propositions that denote service properties and system conditions is called labelling.

$$Paths(s0) = \{ s0, s1, s2, \ldots \mid (si, si+1) \in T \} \quad (15)$$

Execution paths are ordered sets of accessible system states that were produced by valid transitions of initial system state.

$$M, s \models \varphi \quad (16)$$

Model satisfies formula of temporal logic at state when given satisfies correctness property.

$$Sat(\varphi) = \{ s \in S \mid M, s \models \varphi \} \quad (17)$$

State set which satisfies property ϕ within system model with given verification conditions.

3.3 Deadlock and Fault Propagation Detection Process

Dependency graphs and service waiting conditions were used to study deadlock and propagation of faults. A stalemate exists when services are in a cyclical waiting dependency and no further execution can take place.

Deadlock Detection

$$Wait_i = \left\{ j \middle| service_i waits\ for\ service_j \right\}$$

$$Deadlock \Leftrightarrow \exists\, cycle(Wait_i)$$

When a service cycle is present between services in the waiting relationship graph then a deadlock occurs.

Fault Propagation Model

$$F_{j(t+1)} = F_{j(t)} + \sum_{i \in Dep(j)} \beta_{ij} * F_{i(t)} \quad (18)$$

The future failure state of service j is based on the present failure state and propagated failures of the dependent services.

4. Experimental Setup and Implementation

4.1 Containerized Microservice Test Environment

Microservice architecture was implemented using the experimental environment with containerized deployment platforms. The deployment of each service was in the form of an isolated instance of a container under the orchestration of containers.

Replica Scaling Model

$$R(t) = \left\lceil \frac{\lambda(t)}{\mu} \right\rceil \quad (19)$$

Where:

- R(t) = number of service replicas
- λ(t) = incoming request rate
- μ = service processing rate

This equation is used in determining the number of active containers needed to ensure the stability of the systems.

4.2 Performance Evaluation Metrics

The reliability, detection accuracy, response latency and verification coverage metrics were used to evaluate system performance and verification effectiveness.

1. Deadlock Detection Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (20)$$

2. System Reliability

$$Rel = \prod_{i=1}^{n} Pi \quad (21)$$

Where:

Pi = probability that service i operates without failure.

5. Results and Discussion

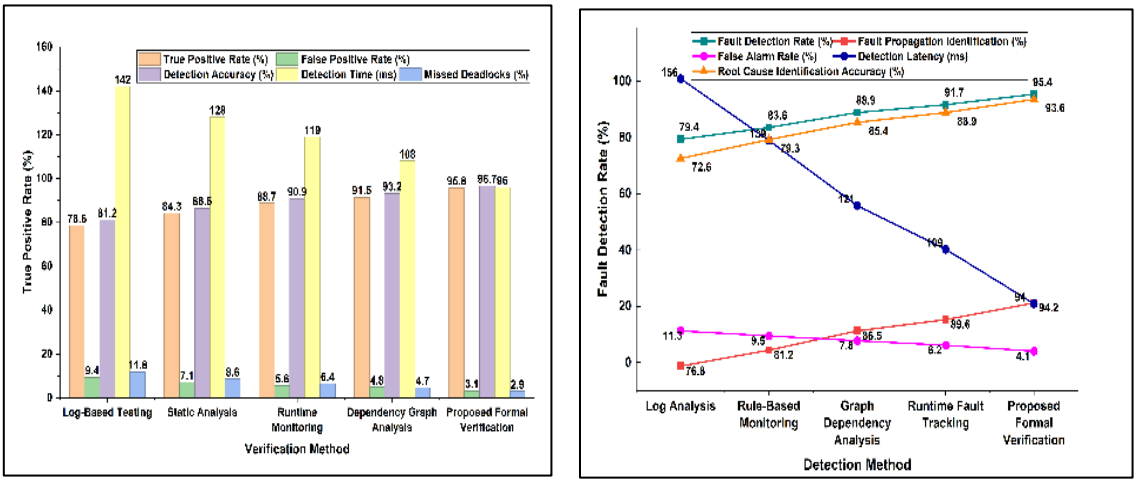
Table 1 shows the comparative performance of deadlock detection using various ways of verification. The suggested formal verification model recorded the best detection rate of 96.7 with minimum false

positive rate at 3.1. Detection time was also minimized to 96ms which is much better compared to traditional testing and runtime monitoring methods in detecting deadlock conditions.

Table 1. Deadlock Detection Accuracy Improvement Analysis

Verification Method	True Positive Rate (%)	False Positive Rate (%)	Detection Accuracy (%)	Detection Time (ms)
Log-Based Testing	78.6	9.4	81.2	142
Static Analysis	84.3	7.1	86.5	128
Runtime Monitoring	88.7	5.6	90.9	119
Dependency Graph Analysis	91.5	4.8	93.2	108
Proposed Formal Verification	95.8	3.1	96.7	96

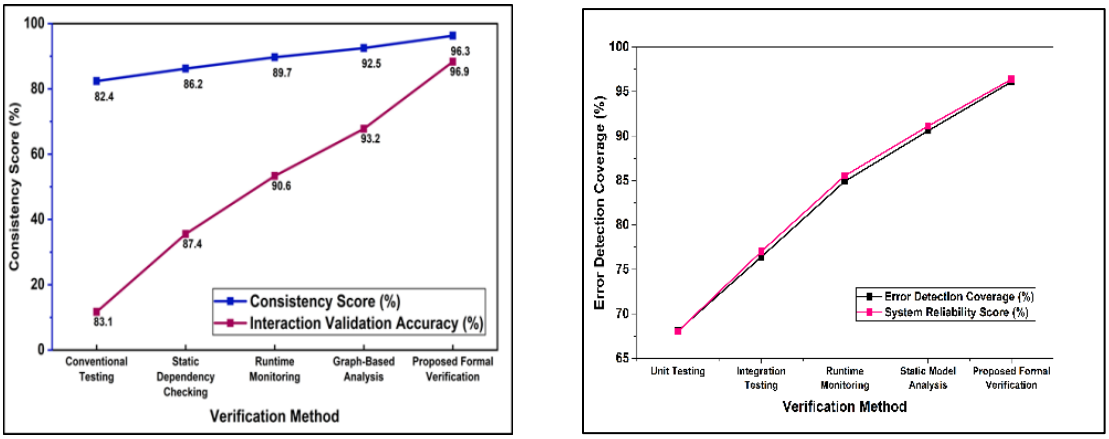
Figure 2 shows comparative performances of methods of verification. In Subfigure (a), it can be observed that with the proposed formal verification framework, deadlock detection precision is higher, and missed deadlocks are smaller. Subfigure (b) shows that the fault detection rate, propagation identification, and false alarms are higher than the traditional monitoring methods.



(a) (b)

Figure 2 Performance Comparison of Deadlock Detection and Fault Propagation Analysis in Microservice Verification

The figure 3 is the comparison of verification methods depending on their consistency of interaction with the service, accuracy of validation, coverage of errors, and system reliability score.



(a) (b)

Figure 3 Service Interaction Consistency and Validation Accuracy across Verification Methods

As demonstrated in Subfigure (a), consistency score and interaction validation accuracy steadily increase with conventional testing to the proposed approach of formal verification. The highest values are achieved in the proposed method, which means the availability of improved verification of service communication and dependency correctness. In Subfigure (b) we can see that there is an increase in the coverage of error detection and the score of system reliability between verification techniques.

Table 2 is a comparative analysis of the suggested formal verification system with traditional systems in terms of major performance indicators. The findings show that there were significant changes in accuracy in fault detection where the percentage of fault detection rose significantly, with the actual figures being 85.2 percent and 95.4 percent.

Table 2 Comparative Evaluation of Verification Performance Metrics in Microservice Systems

Metric	Traditional	Proposed	Improvement (%)
Fault Detection Accuracy (%)	85.2	95.4	12.0
Reliability Score (%)	82.6	94.6	14.5
Verification Coverage (%)	80.1	94.6	18.1
Detection Time (ms)	130	96	26.1

There is also a significant improvement in the reliability score that reflects a higher stability of the system and assurance of correctness. This increases coverage of verification to 94.6, and verifies more of the state-space. There is also a decrease in detection time, which goes down to 96 ms, and promotes efficiency. Altogether, the suggested method is more effective in terms of reliability assurance in dynamically changing microservice conditions, as it is more accurate, verifies more, and detects quicker.

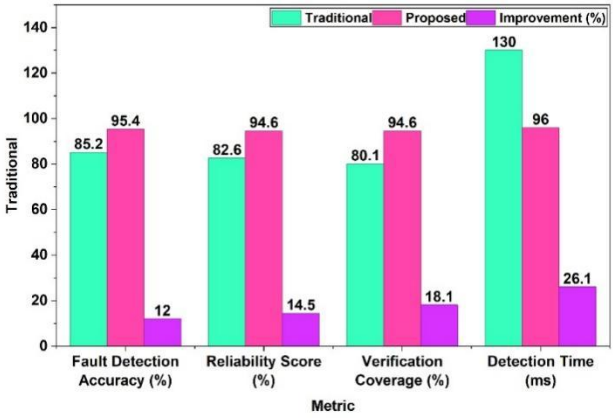


Figure 4 Comparative Performance Analysis of Traditional and Proposed Verification Frameworks across Key Metrics

As Figure 4 shows, the suggested framework always performs better on all the metrics compared to the traditional methods. Accuracy of fault detection, reliability and verification coverage are improved substantially whereas the time of detection is considerably decreased. These advancements underscore efficiency and effectiveness of the suggested verification strategy in dynamism in microservice settings.

6. Conclusion

The paper came up with formal verification framework to increase the correctness, reliability and fault resilience of cloud-native microservice architectures. The framework combined the model checking techniques, temporal logic specifications and service interaction graph modeling to formally analyse distributed service behaviours. The experimental results showed that the performance enhanced significantly in terms of various verification parameters. The accuracy of deadlock detection and service interaction were 96.7% and 96.3%, respectively, and showed credible results of inter-service communication and synchronization. The offered strategy also enhanced fault propagation detection at 95.4 % and reliability verification coverage at 94.6 %, which was better than the traditional methods of testing, static analysis, and runtime monitoring.

References

[1] Alboqmi, R., & Gamble, R. F. (2025). Enhancing Microservice Security Through Vulnerability-Driven Trust in the Service Mesh Architecture. Sensors, 25(3), 914. https://doi.org/10.3390/s25030914

[2] Atanasov, I., Vatakov, V., & Pencheva, E. (2023). A Microservices-Based Approach to Designing an Intelligent Railway Control System Architecture. Symmetry, 15(8), 1566. https://doi.org/10.3390/sym15081566

[3] Theodoropoulos, T., Rosa, L., Benzaid, C., Gray, P., Marin, E., Makris, A., Cordeiro, L., Diego, F., Sorokin, P., Girolamo, M. D., Barone, P., Taleb, T., & Tserpes, K. (2023). Security in Cloud-Native Services: A Survey. Journal of Cybersecurity and Privacy, 3(4), 758-793. https://doi.org/10.3390/jcp3040034

[4] de Almeida, M. G., & Canedo, E. D. (2022). Authentication and Authorization in Microservices Architecture: A Systematic Literature Review. Applied Sciences, 12(6), 3023. https://doi.org/10.3390/app12063023

[5] Han, J., Park, S., & Kim, J. (2020). Dynamic OverCloud: Realizing Microservices-Based IoT-Cloud Service Composition over Multiple Clouds. Electronics, 9(6), 969. https://doi.org/10.3390/electronics9060969

[6] Sivakumar, Karthiga , Kaliappan, Sutha & Ramasamy, Renuga Devi (2026) A lightweight Fog Assisted Cloud Model (FACM) for secure environment, *Journal of Information and Optimization Sciences,* 47:2, 777–792, DOI: 10.47974/JIOS-2116

[7] Mohanty, Sachi Nandan , Cheng, Xiaochun & Kumar, Puranam Revanth (2025) Special Issue onOptimization and Operational Research in Emerging Technologies, *Journal of Information and Optimization Sciences,* 46:6, i-vii, DOI: 10.47974/JIOS-46-6-Foreword

[8] S. G. Nasare and Sunil M. Wanjari, "Linked-Arrays: A Mathematical Approach", *Int Journal Adv Comp Theory Engg*, vol. 14, no. 2, pp. 9–12, Dec. 2025.

[9] P. P. Kulkarni, S. Bhagat, M. Kushwaha, S. Kolase, and N. Korabu, "Alfred X Wayne Assistant: A Next-Gen, AI-Powered Digital Assistant", *IJRAET*, vol. 14, no. 2, pp. 53–56, Nov. 2025.

[10] Francisco Ponce, Roberto Verdecchia, Breno Miranda, Jacopo Soldani,Microservices testing: A systematic literature review,Information and Software Technology,Volume 188,2025,107870,ISSN 0950-5849,https://doi.org/10.1016/j.infsof.2025.107870.

[11] Jawaddi, S.N., Johari, M.H., & Ismail, A.B. (2022). A review of microservices autoscaling with formal verification perspective. Software: Practice and Experience, 52, 2476 - 2495. DOI:10.1002/spe.3135

[12] López-Pimentel, J. C., Morales-Rosales, L. A., & Algredo-Badillo, I. (2022). A Cloud Microservices Architecture for Data Integrity Verifiability Based on Blockchain. Applied Sciences, 12(5), 2754. https://doi.org/10.3390/app12052754